Employee Management System

Database Schema Documentation

Total Tables	12
Database Type	Relational (SQL / MySQL)
Core Entities	Departments, Positions, Employees
Relationship Tables	Salaries, Attendance, Leave Types, Leave Requests, Payroll, Bonuses, Performance Reviews, Projects, Employee Projects

Table of Contents

1. Overview & Entity Summary

This document describes the complete relational schema for the Employee Management System (EMS). The database consists of 12 tables split into two categories: Core Entity Tables (Departments, Positions, Employees) that hold master records, and Relationship / Transactional Tables (Salaries, Attendance, Leave Types, Leave Requests, Payroll, Bonuses, Performance Reviews, Projects, Employee Projects) that connect core entities and record activity over time.

#	Table Name	Purpose	Type
1	Departments	Organizational departments and budgets	Core
2	Positions	Job titles with salary bands	Core
3	Employees	Employee personal, job & reporting details	Core
4	Salaries	Salary amount history per employee	Relationship
5	Attendance	Daily attendance records per employee	Relationship
6	Leave Types	Categories of leave and annual limits	Relationship
7	Leave Requests	Leave applications and approval status	Relationship
8	Payroll	Monthly pay run per employee	Relationship
9	Bonuses	One-off bonus payments per employee	Relationship
10	Performance Reviews	Reviewer ratings & comments per employee	Relationship
11	Projects	Projects run within a department	Relationship
12	Employee Projects	Links employees to the projects they work on	Relationship

2. Departments

Stores organizational departments such as Human Resources, IT, and Finance, along with location and budget. This is a core master table referenced by Employees and Projects.

Column Structure

Column	Type	Key	Description
dept_id	INT	PK	Unique identifier, auto-increment
dept_name	VARCHAR(100)	-	Name of the department
location	VARCHAR(100)	-	City where the department is based
budget	DECIMAL(15,2)	-	Allocated annual budget

SQL Definition

```
CREATE TABLE departments (
    dept_id INT AUTO_INCREMENT PRIMARY KEY,
    dept_name VARCHAR(100) NOT NULL,
    location VARCHAR(100),
    budget DECIMAL(15,2)
);
```

Foreign Key Notes: None (this is a top-level parent table).

3. Positions

Defines job titles available across the organization, each with a minimum and maximum salary band used for compensation planning.

Column Structure

Column	Type	Key	Description
position_id	INT	PK	Unique identifier, auto-increment
title	VARCHAR(100)	-	Job title (e.g. Software Engineer)
min_salary	DECIMAL(10,2)	-	Minimum salary for the position
max_salary	DECIMAL(10,2)	-	Maximum salary for the position

SQL Definition

```
CREATE TABLE positions (
    position_id INT AUTO_INCREMENT PRIMARY KEY,
    title VARCHAR(100) NOT NULL,
    min_salary DECIMAL(10,2),
    max_salary DECIMAL(10,2)
);
```

Foreign Key Notes: None (this is a top-level parent table).

4. Employees

Stores personal, contact, and employment information for every employee. Includes a self-referencing manager_id so the reporting hierarchy can be modeled within the same table.

Column Structure

Column	Type	Key	Description
emp_id	INT	PK	Unique identifier, auto-increment
first_name / last_name	VARCHAR(50)	-	Employee's name
email	VARCHAR(100)	-	Contact email (unique)
phone	VARCHAR(15)	-	Contact phone number
hire_date	DATE	-	Date the employee was hired
dept_id	INT	FK	References departments(dept_id)
manager_id	INT	FK	References employees(emp_id), self
position_id	INT	FK	References positions(position_id)
job_title	VARCHAR(100)	-	Displayed job title
status	ENUM	-	Active / Inactive / Terminated

SQL Definition

```
CREATE TABLE employees (
    emp_id INT AUTO_INCREMENT PRIMARY KEY,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    phone VARCHAR(15),
    hire_date DATE NOT NULL,
    dept_id INT,
    manager_id INT,
    position_id INT,
    job_title VARCHAR(100),
    status ENUM('Active','Inactive','Terminated') DEFAULT 'Active',
    FOREIGN KEY (dept_id) REFERENCES departments(dept_id),
    FOREIGN KEY (manager_id) REFERENCES employees(emp_id),
    FOREIGN KEY (position_id) REFERENCES positions(position_id)
);
```

Foreign Key Notes: dept_id -> departments.dept_id, manager_id -> employees.emp_id (self), position_id -> positions.position_id. Departments and Positions must exist before Employees.

5. Salaries

Maintains a history of salary amounts assigned to an employee over time, each tied to an effective date.

Column Structure

Column	Type	Key	Description
salary_id	INT	PK	Unique identifier, auto-increment
emp_id	INT	FK	References employees(emp_id)
amount	DECIMAL(10,2)	-	Salary amount
effective_date	DATE	-	Date this salary took effect
currency	VARCHAR(10)	-	Currency code, default INR

SQL Definition

```
CREATE TABLE salaries (
    salary_id INT AUTO_INCREMENT PRIMARY KEY,
    emp_id INT NOT NULL,
    amount DECIMAL(10,2) NOT NULL,
    effective_date DATE NOT NULL,
    currency VARCHAR(10) DEFAULT 'INR',
    FOREIGN KEY (emp_id) REFERENCES employees(emp_id)
);
```

Foreign Key Notes: emp_id -> employees.emp_id. Employees table must exist before Salaries.

6. Attendance

Records daily attendance status (Present / Absent / Half Day / Leave) for each employee.

Column Structure

Column	Type	Key	Description
attendance_id	INT	PK	Unique identifier, auto-increment
emp_id	INT	FK	References employees(emp_id)
date	DATE	-	Date of the attendance record
check_in / check_out	TIME	-	Clock-in and clock-out times
status	ENUM	-	Present / Absent / Half Day / Leave

SQL Definition

```
CREATE TABLE attendance (
    attendance_id INT AUTO_INCREMENT PRIMARY KEY,
    emp_id INT NOT NULL,
    date DATE NOT NULL,
    check_in TIME,
    check_out TIME,
    status ENUM('Present','Absent','Half Day','Leave') DEFAULT 'Present',
    FOREIGN KEY (emp_id) REFERENCES employees(emp_id),
    UNIQUE KEY unique_emp_date (emp_id, date)
);
```

Foreign Key Notes: emp_id -> employees.emp_id. UNIQUE(emp_id, date) prevents duplicate same-day entries.

7. Leave Types

Defines categories of leave (Casual, Sick, Earned, Maternity, Paternity, Unpaid) and the maximum days allowed per year.

Column Structure

Column	Type	Key	Description
leave_type_id	INT	PK	Unique identifier, auto-increment
leave_name	VARCHAR(50)	-	Name of the leave category
max_days_per_year	INT	-	Maximum days allowed per year

SQL Definition

```
CREATE TABLE leave_types (
    leave_type_id INT AUTO_INCREMENT PRIMARY KEY,
    leave_name VARCHAR(50) NOT NULL,
    max_days_per_year INT DEFAULT 0
);
```

Foreign Key Notes: None (this is a top-level parent table).

8. Leave Requests

Stores leave applications submitted by employees, the leave type requested, approval status, and who approved it.

Column Structure

Column	Type	Key	Description
leave_id	INT	PK	Unique identifier, auto-increment
emp_id	INT	FK	References employees(emp_id)
leave_type_id	INT	FK	References leave_types(leave_type_id)
start_date / end_date	DATE	-	Leave period
status	ENUM	-	Pending / Approved / Rejected
approved_by	INT	FK	References employees(emp_id)

SQL Definition

```
CREATE TABLE leave_requests (
    leave_id INT AUTO_INCREMENT PRIMARY KEY,
    emp_id INT NOT NULL,
    leave_type_id INT NOT NULL,
    start_date DATE NOT NULL,
    end_date DATE NOT NULL,
    status ENUM('Pending','Approved','Rejected') DEFAULT 'Pending',
    approved_by INT,
    FOREIGN KEY (emp_id) REFERENCES employees(emp_id),
    FOREIGN KEY (leave_type_id) REFERENCES leave_types(leave_type_id),
    FOREIGN KEY (approved_by) REFERENCES employees(emp_id)
);
```

Foreign Key Notes: emp_id -> employees.emp_id, leave_type_id -> leave_types.leave_type_id, approved_by -> employees.emp_id. Employees and Leave Types must exist first.

9. Payroll

Tracks the monthly pay run for each employee, including basic pay, deductions, and net pay.

Column Structure

Column	Type	Key	Description
payroll_id	INT	PK	Unique identifier, auto-increment
emp_id	INT	FK	References employees(emp_id)
month / year	INT	-	Pay period (month 1-12)
basic_pay	DECIMAL(10,2)	-	Gross pay before deductions
deductions	DECIMAL(10,2)	-	Total deductions
net_pay	DECIMAL(10,2)	-	Final amount paid

SQL Definition

```
CREATE TABLE payroll (
    payroll_id INT AUTO_INCREMENT PRIMARY KEY,
    emp_id INT NOT NULL,
    month INT NOT NULL CHECK (month BETWEEN 1 AND 12),
    year INT NOT NULL,
    basic_pay DECIMAL(10,2) NOT NULL,
    deductions DECIMAL(10,2) DEFAULT 0,
    net_pay DECIMAL(10,2) NOT NULL,
    FOREIGN KEY (emp_id) REFERENCES employees(emp_id),
    UNIQUE KEY unique_emp_month_year (emp_id, month, year)
);
```

Foreign Key Notes: emp_id -> employees.emp_id. UNIQUE(emp_id, month, year) prevents duplicate pay runs.

10. Bonuses

Records one-off bonus payments awarded to employees, along with the reason and date awarded.

Column Structure

Column	Type	Key	Description
bonus_id	INT	PK	Unique identifier, auto-increment
emp_id	INT	FK	References employees(emp_id)
amount	DECIMAL(10,2)	-	Bonus amount
reason	VARCHAR(255)	-	Reason the bonus was awarded
date_awarded	DATE	-	Date the bonus was given

SQL Definition

```
CREATE TABLE bonuses (
    bonus_id INT AUTO_INCREMENT PRIMARY KEY,
    emp_id INT NOT NULL,
    amount DECIMAL(10,2) NOT NULL,
    reason VARCHAR(255),
    date_awarded DATE NOT NULL,
    FOREIGN KEY (emp_id) REFERENCES employees(emp_id)
);
```

Foreign Key Notes: emp_id -> employees.emp_id. Employees table must exist before Bonuses.

11. Performance Reviews

Stores periodic performance ratings and comments given to an employee by a reviewer (typically their manager).

Column Structure

Column	Type	Key	Description
review_id	INT	PK	Unique identifier, auto-increment
emp_id	INT	FK	References employees(emp_id)
reviewer_id	INT	FK	References employees(emp_id)
review_date	DATE	-	Date of the review
rating	DECIMAL(3,1)	-	Rating between 1 and 5
comments	TEXT	-	Reviewer's written comments

SQL Definition

```
CREATE TABLE performance_reviews (
    review_id INT AUTO_INCREMENT PRIMARY KEY,
    emp_id INT NOT NULL,
    reviewer_id INT,
    review_date DATE NOT NULL,
    rating DECIMAL(3,1) CHECK (rating BETWEEN 1 AND 5),
    comments TEXT,
    FOREIGN KEY (emp_id) REFERENCES employees(emp_id),
    FOREIGN KEY (reviewer_id) REFERENCES employees(emp_id)
);
```

Foreign Key Notes: emp_id -> employees.emp_id, reviewer_id -> employees.emp_id (self). Employees table must exist first.

12. Projects

Lists projects run within a department, with a start and end date.

Column Structure

Column	Type	Key	Description
project_id	INT	PK	Unique identifier, auto-increment
project_name	VARCHAR(150)	-	Name of the project
start_date / end_date	DATE	-	Project duration
dept_id	INT	FK	References departments(dept_id)

SQL Definition

```
CREATE TABLE projects (
    project_id INT AUTO_INCREMENT PRIMARY KEY,
    project_name VARCHAR(150) NOT NULL,
    start_date DATE,
    end_date DATE,
    dept_id INT,
    FOREIGN KEY (dept_id) REFERENCES departments(dept_id)
);
```

Foreign Key Notes: dept_id -> departments.dept_id. Departments table must exist before Projects.

13. Employee Projects

A many-to-many mapping table connecting employees to the projects they are assigned to, along with their role and allocated hours.

Column Structure

Column	Type	Key	Description
emp_id / project_id	INT	PK	Composite primary key
role_in_project	VARCHAR(100)	-	Employee's role on the project
hours_allocated	INT	-	Hours allocated to the project

SQL Definition

```
CREATE TABLE employee_projects (
    emp_id INT NOT NULL,
    project_id INT NOT NULL,
    role_in_project VARCHAR(100),
    hours_allocated INT DEFAULT 0,
    PRIMARY KEY (emp_id, project_id),
    FOREIGN KEY (emp_id) REFERENCES employees(emp_id),
    FOREIGN KEY (project_id) REFERENCES projects(project_id)
);
```

Foreign Key Notes: emp_id -> employees.emp_id, project_id -> projects.project_id. Both parent tables must exist first.

14. Relationship Diagram (Text Form)

The diagram below summarizes how the 12 tables connect to one another via foreign keys.

```
Departments -----> Employees (dept_id)
-----> Projects (dept_id)

Positions -----> Employees (position_id)

Employees -----> Employees (manager_id, self)
-----> Salaries (emp_id)
-----> Attendance (emp_id)
-----> Leave Requests (emp_id, approved_by)
-----> Payroll (emp_id)
-----> Bonuses (emp_id)
-----> Performance Reviews (emp_id, reviewer_id)
-----> Employee Projects (emp_id)

Leave Types -----> Leave Requests (leave_type_id)

Projects -----> Employee Projects (project_id)
```

15. Table Creation Order

Because of foreign key dependencies, tables must be created in the order below to avoid 'reference does not exist' errors. Parent tables always come before the child tables that reference them.

Step	Table	Depends On
1	Departments	None
2	Positions	None
3	Leave Types	None
4	Employees	Departments, Positions, Employees (self)
5	Salaries	Employees
6	Attendance	Employees
7	Leave Requests	Employees, Leave Types
8	Payroll	Employees
9	Bonuses	Employees
10	Performance Reviews	Employees
11	Projects	Departments
12	Employee Projects	Employees, Projects

Document generated for the Employee Management System project. This schema reflects the 12-table design covering departments, positions, employees, salaries, attendance, leave types, leave requests, payroll, bonuses, performance reviews, projects, and employee projects.